

INSTITUTO CATÓLICO KAROL WOJTYŁA

Actividad desarrollada:

DOCUMENTO DE EVIDENCIAS – SEMANA 2

Grado:

Tercer Año de Bachillerato Técnico en Desarrollo de Software

MÓDULO 3.3: Administración de Bases de Datos

Alumno:

José Rolando Velasco Peña

Docente:

Lic. Criseyda Guadalupe Araujo Mengar

Cabañas Oeste, El Salvador

18 de septiembre de 2026

ÍNDICE

ÍNDICE	2
TABLA DE FIGURAS	3
1. INTRODUCCIÓN	4
2. OBJETIVOS DE LA SEMANA	5
3. DESCRIPCIÓN DEL PROYECTO Y DISTRIBUCIÓN DE CARGOS	6
4. ORGANIZACIÓN DEL TRABAJO	7
4.1. Base de datos utilizada	7
4.2. Responsabilidad Individual.....	9
4.3. Rama utilizada	9
5. ADMINISTRACIÓN DE DATOS CRUD	9
5.1. INSERT: Registro de datos de prueba	10
5.2. Comprobación de registros	10
5.3. CONSULTAS: CLASE UNO	11
6. CONSULTAS SQL	17
7. TRABAJO COLABORATIVO CON GITHUB.....	21
7.1 Repositorio.....	21
7.2 Rama individual	21
8. COMMITS	22
9. INTEGRACIÓN DEL APORTE	23
9.1. Historial de COMMITS.....	23
9.2. Archivos modificados	24
10. REFLEXIÓN INDIVIDUAL	25
11. CONCLUSIÓN	26

TABLA DE FIGURAS

Figura 1 Captura de la base de datos utilizada para las prácticas de la semana.....	8
Figura 2 Captura de las tablas existentes en la DB de SafeSchool Alert.....	9
Figura 3 Sintaxis SQL utilizada para ingresar los registros de prueba de alertas	10
Figura 4 Captura de consulta SELECT de la tabla Alertas	11
Figura 5 Consulta SELECT de la tabla Alertas, para la práctica de clase 1.....	11
Figura 6 Consulta SELECT utilizado LEFT y INNER JOIN, para la práctica de clase 1.....	12
Figura 7 Consulta SELECT utilizado WHERE, para la práctica de clase 1	13
Figura 8 Consulta UPDATE, para modificar estados de emergencia.....	13
Figura 9 Consulta de verificación de datos actualizados de UPDATE.	13
Figura 10 Consulta UPDATE para modificar el usuario de una alerta	14
Figura 11 Consulta de verificación de datos actualizados	14
Figura 12 Consulta DELETE para la eliminación de alerta con estado Falsa	14
Figura 13 Consulta de verificación de DELETE.....	15
Figura 14 Consulta SELECT de la tabla Ubicaciones.....	15
Figura 15 Consulta de SELECT utilizando la condición WHERE.....	16
Figura 16 Consulta de SELECT utilizando WHERE IN().	16
Figura 17 Consulta SELECT utilizando ORDER BY para ordenar alertas	17
Figura 18 Consulta SELECT utilizando LIKE.....	18
Figura 19 Consulta para mostrar alertas por ubicación.	18
Figura 20 Consulta para obtener registros de tipo "Incendio"......	19
Figura 21 Consulta SELECT utilizando GROUP BY.....	20
Figura 22 Consulta SELECT para mostrar cantidad de emergencias por tipo con GROUP BY.	20
Figura 23 Captura del repositorio utilizado en GitHub	21
Figura 24 Captura de rama utilizada "Rolando_SafeSchool_DB".	22
Figura 25 Captura de evidencia del Historial de COMMITS realizado	23
Figura 26 Capturad de evidencia de integración del mi trabajo a la rama mian	24
Figura 27 Captura de archivos modificados	24

1. INTRODUCCIÓN

A lo largo del presente documento se desarrollan las actividades correspondientes a esta semana. Durante esta práctica se trabajó con la base de datos de SafeSchool Alert, realizando diferentes consultas SQL para obtener y organizar la información de las emergencias registradas. También se utilizaron sentencias como SELECT, JOIN y GROUP BY para relacionar y mostrar los datos de una mejor manera.

Por ello, a lo largo de estas dos semanas se ha venido trabajando en una estructura de base de datos funcional y robusta, conectando las diferentes tablas por medio de relaciones y utilizando claves foráneas para obtener información más completa y útil de cada emergencia, con el propósito de gestionar adecuadamente cada situación. Además, durante esta semana se trabajó de forma colaborativa utilizando GitHub, “realizando commits, trabajando con ramas y utilizando un Pull Request para agregar los cambios realizados.

2. OBJETIVOS DE LA SEMANA

2.1. Objetivo general

Diseñar una estructura funcional y robusta de base de datos para el sistema SafeSchool Alert, capaz de almacenar, organizar y gestionar de manera eficiente la información utilizada por la aplicación móvil, para atender gestionar y dándole un seguimiento detallado a cada emergencia de tectada en la institución educativa.

2.2. Objetivos específicos

- Crear una estructura funcional para la base de datos, trabajando de forma colaborativa con los integrantes del equipo de SafeSchool Alert, utilizando herramientas de desarrollo y colaboración como Git y GitHub.
- Aprender y aplicar consultas CRUD para la manipulación de datos mediante sentencias SQL, con el propósito de atender necesidades reales que la aplicación móvil requiere para la gestión y atención rápida de emergencias dentro de la institución educativa.

3. DESCRIPCIÓN DEL PROYECTO Y DISTRIBUCIÓN DE CARGOS

3.1. Descripción del proyecto

SafeSchool Alert es un sistema inteligente de alertas y emergencias escolares, diseñado para facilitar la detección, comunicación y seguimiento de diferentes situaciones de emergencia que puedan presentarse dentro de la institución educativa. Su propósito principal es contribuir a una respuesta rápida y organizada, ayudando a proteger la vida y seguridad de estudiantes, docentes y personal administrativo.

3.2. Distribución de cargos de la base de datos

En esta sección se presenta la distribución de cargos de cada integrante que conforma el equipo de SafeSchool Alert. Esta organización fue establecida con el propósito de asignar las funciones de administración de la base de datos de forma equitativa y eficiente de la administración de la base de datos, de forma equitativa y eficiente para desarrollar las actividades prácticas y de preparación de nuestra base de datos correspondiente a la semana dos. Para ello en la siguiente tabla se presenta la distribución de cargos y responsabilidades que se han asignado a cada integrante del equipo:

3.2.1. Tabla de cargos del equipo

N.º	Integrante	Tabla asignada	Responsabilidad
1	José Rolando Velasco Peña	alertas	Encargado de crear y gestionar la tabla alertas, simulando registros de prueba de las posibles emergencias que puedan ocurrir en la institución. También se encargará de diseñar sentencias SQL capaces de obtener datos completos y útiles que posteriormente serán mostrados al usuario.
2	Luis Mario Meléndez Escobar	historial	Encargado de crear y gestionar la simulación de registros de prueba relacionados con los eventos de emergencia de la tabla historial, donde se llevará un seguimiento de todas las acciones realizadas sobre una emergencia, desde su activación hasta su solución.

3	Ulises de Jesús Mercado Alberto	usuarios	Encargado de gestionar los usuarios autorizados para administrar y dar seguimiento a cada emergencia detectada. También se encargará de diseñar las sentencias SQL necesarias para validar y autenticar a cada usuario registrado en el sistema.
---	------------------------------------	----------	--

Cabe recalcar que el sistema cuenta con otras tablas. Sin embargo, la razón por la que no se seleccionó una de las tablas restantes para cada integrante es que las siguientes tablas:

- **Tabla:** instituciones
- **Tabla:** tipos_emergencias
- **Tabla:** ubicaciones

Almacenan datos iniciales que, por el momento, no necesitan de más registros, lo que dificulta utilizarlas para cumplir con las actividades que solicita esta semana dos, ya que la institución con la que se trabajará será únicamente una. Esta cuenta con tres ubicaciones (edificios), y actualmente el sistema solo detecta y permite dar seguimiento a emergencias de tipo incendio, médica y seguridad.

4. ORGANIZACIÓN DEL TRABAJO

4.1. Base de datos utilizada

La base de datos utilizada en las actividades prácticas corresponde a la base de datos creada y diseñada durante la semana uno la cual fue nombrada: “safeschool_alert”, utilizando PostgreSQL. Cada tabla mencionada en este documento corresponden a las tablas creadas anteriormente, donde se realizó la creación de cada una de las tablas y sus relaciones correspondientes, para poder trabajar y almacenar datos dentro de cada una.

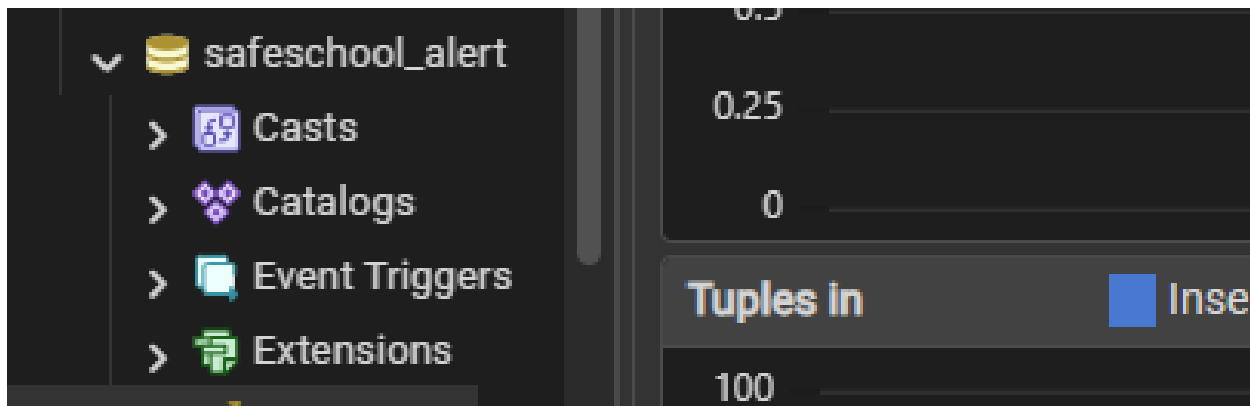


Figura 1 Captura de la base de datos utilizada para las prácticas de la semana

Las tablas utilizadas durante esta práctica se mantienen para el desarrollo de las actividades prácticas y la preparación de la estructura de la base de datos. Como parte de las nuevas actualizaciones realizadas, se creó la tabla “historial”, la cual fue diseñada para registrar cada evento o modificación aplicada a las emergencias detectadas por el sistema. De esta manera, se podrá llevar un seguimiento ordenado de las acciones realizadas desde la activación de una emergencia hasta su solución.

N.º	Tabla	Responsabilidad
1	instituciones	Almacena la información de las instituciones educativas registradas.
2	usuarios	Almacena los usuarios asociados a una institución.
3	ubicaciones	Registra las diferentes ubicaciones dentro de una institución.
4	tipos_emergencia	Contiene los tipos de emergencia que puede manejar el sistema.
5	alertas	Registra las emergencias y alertas generadas dentro de la institución
6	historial	Conserva el registro histórico de las alertas y eventos gestionados.

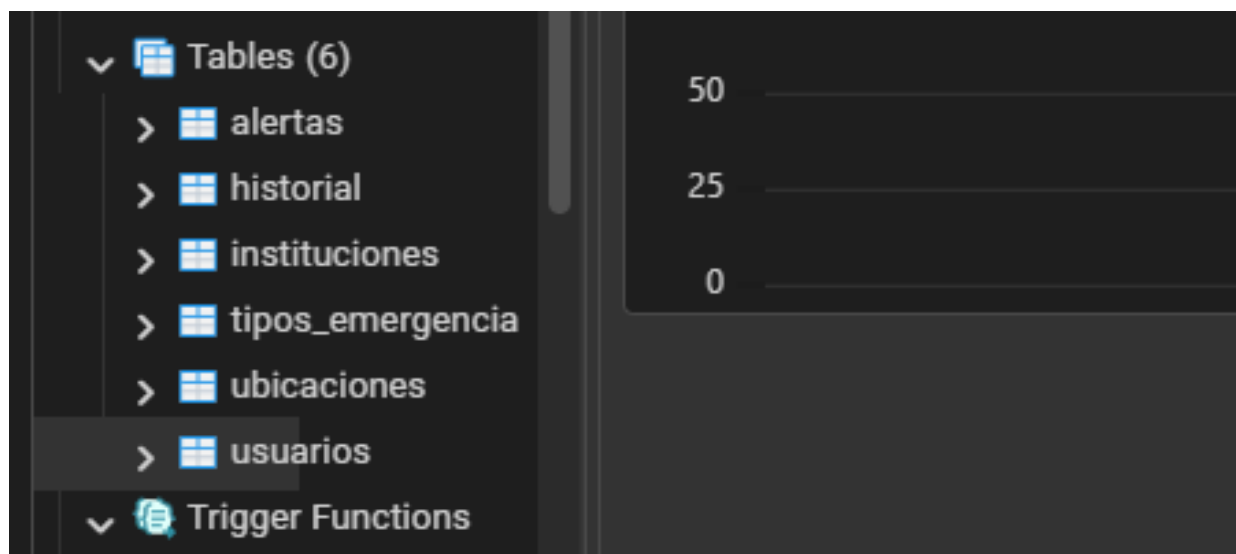


Figura 2 Captura de las tablas existentes en la DB de SafeSchool Alert

4.2. Responsabilidad Individual

Las responsabilidades que se me asignaron al dialogarlo en equipo fueron las de la gestión de la tabla “*alertas*”, la cual será con la que se utilizara para realizar cada ejercicio presentado en el presente documento, ya que fue la tabla que se asignó para trabajar, al distribuir las responsabilidades de cada integrante.

4.3. Rama utilizada

La rama creada para trabajar en el repositorio del proyecto SafeSchool Alert será “Rolando_SafeSchool_DB”. En ella se agregarán todos los aportes y trabajos realizados, así como las mejoras y modificaciones relacionadas con la base de datos. De esta manera, se podrá mantener organizado el trabajo realizado y aportar los avances correspondientes al equipo que forma parte del proyecto.

5. ADMINISTRACIÓN DE DATOS CRUD

A continuación, se presenta las sentencias y los resultados de las actividades prácticas propuestas en cada guía de la semana, las cuales se abarcan las cuatro operaciones básicas para gestionar datos en una base de datos: Crear (INSERT), Leer (SELECT), Actualizar (UPDATE) y Borrar (DELETE).

5.1. INSERT: Registro de datos de prueba

Teniendo en cuenta los las responsabilidades, asignadas para trabajar en esta semana se procedió: Lo primero fue registrar 7 registros prueba, simulando emergencias registradas por el sistema y reportadas por la institución educativa.

Para ello se usó de la operación: INSERT, para registrar los registros correspondientes, para lograr una simulación realista se asignaron valores a cada campo con datos reales que la aplicación pudiera almacenar de cada emergencia, con dñentes estados y ubicaciones cada una.

```
-- 1.1. INSERT:
-- 1.1.1. CONSULTA 1: Registro de 7 posibles emergencias que pudieran suceder en la institución educativa.

INSERT INTO public.alertas (
  id_usuario,
  id_tipo,
  id_ubicacion,
  comando_recibido,
  descripcion, estado,
  hora_emerg,
  fecha)
VALUES
(1, 1, 1, 'Incendio,Edif_A,Activa', 'Humo detectado cerca del laboratorio principal en Edificio A', 'ACTIVA', '10:30:00', '2026-09-24'),
(2, 2, 2, 'Medica,Edif_B,Activa', 'Estudiante herido en la cancha central del Edificio B', 'ACTIVA', '14:30:00', '2026-09-24'),
(3, 3, 3, 'Seguridad,Edif_C,Activa', 'Persona no autorizada detectada en el acceso al Edificio C', 'EN_PROCESO', '11:45:00', '2026-09-24'),
(4, 1, 1, 'Incendio,Edif_A,Activa', 'Alarma de incendio activada en pasillo del Edificio A', 'ATENDIDA', '12:20:00', '2026-09-24'),
(3, 2, 1, 'Medica,Edif_A,Activa', 'Desmayo de alumno en el primero Sección "A" del Edificio A', 'FALSA', '15:10:00', '2026-09-24'),
(1, 1, 2, 'Incendio,Edif_B,Activa', 'Humo detectado cerca del laboratorio principal en Edificio B', 'ATENDIDA', '15:10:00', '2026-09-24'),
(3, 2, 2, 'Medica,Edif_B,Activa', 'Estudiante lesionado en la cancha.', 'FALSA', '15:10:00', '2026-08-24');
```

Figura 3 Sintaxis SQL utilizada para ingresar los registros de prueba de alertas

5.2. Comprobación de registros

Una vez realizada la ejecución de los registros correctamente, para comprobar que los registros se registraron sin problemas: se procedió a realizar una consulta de datos con la operación SQL: SELECT.

Gracias a esta es posible obtener cada uno de los registros existentes dentro en la tabla alertas. La prueba fue exitosa mostrando los 7 registros de prueba creados anteriormente con INSERT.

Query

Query History

26

27

28

29

SELECT * FROM public.alertas;

Data Output

Messages

Notifications

<

Figura 4 Captura de consulta SELECT de la tabla Alertas

5.3. CONSULTAS: CLASE UNO

5.3.1. Consulta 1: Utilizando SELECT, mostrar todas las emergencias registradas en la tabla alertas.

Para realizar este ejerció se hizo uso de SELECT para obtener cada una de las emergencias registradas. De este modo es posible visualizar cuales son las emergencias existentes dentro de la institución educativa. La consulta mostro un total de 7 registros de emergencias registradas.

Query

Query History

25

26

27

28

SELECT * FROM public.alertas;

Data Output

Messages

Notifications

Showing rows: 1 to 7

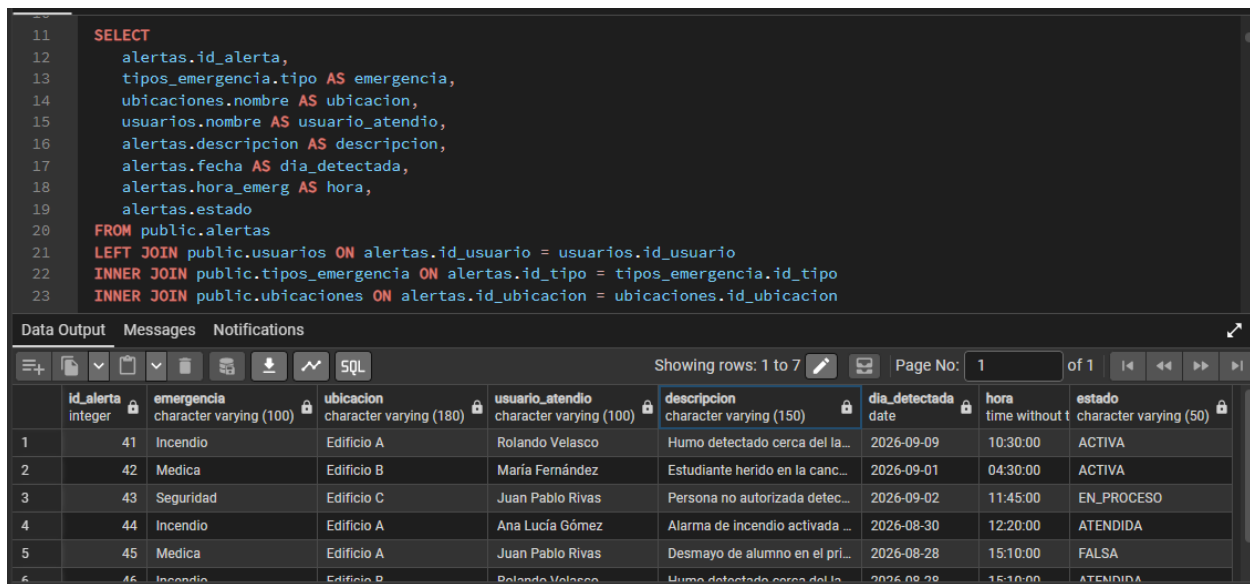
Page No: 1 of 1

	id_alerta [PK] integer	id_usuario integer	id_tipo integer	id_ubicacion integer	comando_recibido character varying (50)	descripcion character varying (150)	estado character varying (50)	hora_emerg time without time zone	fecha date
1	41	1	1	1	Incendio,Edif_A,Activa	Humo detectado cerca del labor...	ACTIVA	10:30:00	2026-09...
2	42	2	2	2	Medica,Edif_B,Activa	Estudiante herido en la cancha c...	ACTIVA	04:30:00	2026-09...
3	43	3	3	3	Seguridad,Edif_C,Activa	Persona no autorizada detectad...	EN_PROCESO	11:45:00	2026-09...
4	44	4	1	1	Incendio,Edif_A,Activa	Alarma de incendio activada en ...	ATENDIDA	12:20:00	2026-08...
5	45	3	2	1	Medica,Edif_A,Activa	Desmayo de alumno en el primer...	FALSA	15:10:00	2026-08...
6	46	1	1	2	Incendio,Edif_B,Activa	Humo detectado cerca del labor...	ATENDIDA	15:10:00	2026-08...
7	47	3	2	2	Medica,Edif_B,Activa	Estudiante lesionado en la canch...	FALSA	05:10:00	2026-08...

Figura 5 Consulta SELECT de la tabla Alertas, para la práctica de clase 1.

5.3.2. Consulta 2: Utilizando SELECT, realizar un LEFT y un INNER JOIN para obtener datos más completos de cada emergencia.

Para realizar esta consulta fue necesario utilizar los métodos de unión de tablas LEFT JOIN para obtener todos los registros de las emergencias, y INNER JOIN para obtener la categoría, ubicación donde se detectó la emergencia, así como obtener el usuario que atendió la emergencia



```
11 SELECT
12     alertas.id_alerta,
13     tipos_emergencia.tipo AS emergencia,
14     ubicaciones.nombre AS ubicacion,
15     usuarios.nombre AS usuario_atendio,
16     alertas.descripcion AS descripcion,
17     alertas.fecha AS dia_detectada,
18     alertas.hora_emerg AS hora,
19     alertas.estado
20 FROM public.alertas
21 LEFT JOIN public.usuarios ON alertas.id_usuario = usuarios.id_usuario
22 INNER JOIN public.tipos_emergencia ON alertas.id_tipo = tipos_emergencia.id_tipo
23 INNER JOIN public.ubicaciones ON alertas.id_ubicacion = ubicaciones.id_ubicacion
```

Data Output Messages Notifications

Showing rows: 1 to 7 Page No: 1 of 1

	id_alerta integer	emergencia character varying (100)	ubicacion character varying (180)	usuario_atendio character varying (100)	descripcion character varying (150)	dia_detectada date	hora time without t	estado character varying (50)
1	41	Incendio	Edificio A	Rolando Velasco	Humo detectado cerca del la...	2026-09-09	10:30:00	ACTIVA
2	42	Medica	Edificio B	María Fernández	Estudiante herido en la canc...	2026-09-01	04:30:00	ACTIVA
3	43	Seguridad	Edificio C	Juan Pablo Rivas	Persona no autorizada detec...	2026-09-02	11:45:00	EN_PROCESO
4	44	Incendio	Edificio A	Ana Lucía Gómez	Alarma de incendio activada ...	2026-08-30	12:20:00	ATENDIDA
5	45	Medica	Edificio A	Juan Pablo Rivas	Desmayo de alumno en el pri...	2026-08-28	15:10:00	FALSA
6	46	Incendio	Edificio B	Rolando Velasco	Humo detectado cerca del la...	2026-08-28	15:10:00	ATENDIDA

Figura 6 Consulta SELECT utilizado LEFT y INNER JOIN, para la práctica de clase 1.

5.3.3. Consulta 3: Utilizando SELECT, mostrar todas las emergencias activas dentro de la institución educativa.

Para realizar esta consulta, se hace uso de las sentencias SQL: LEFT y INNER JOIN, para obtener datos más complejos, pero con una condición WHERE, para mostrar solo las emergencias donde su estado sea igual a “activa”.

```

28 SELECT
29     alertas.id_alerta,
30     tipos_emergencia.tipo AS emergencia,
31     ubicaciones.nombre AS ubicacion,
32     usuarios.nombre AS usuario_atendio,
33     alertas.descripcion AS descripcion,
34     alertas.fecha AS dia_detectada,
35     alertas.hora_emerg AS hora,
36     alertas.estado
37
38 FROM public.alertas
39 LEFT JOIN public.usuarios ON alertas.id_usuario = usuarios.id_usuario
40 INNER JOIN public.tipos_emergencia ON alertas.id_tipo = tipos_emergencia.id_tipo
41 INNER JOIN public.ubicaciones ON alertas.id_ubicacion = ubicaciones.id_ubicacion
42 WHERE alertas.estado = 'ACTIVA';

```

	id_alerta	emergencia	ubicacion	usuario_atendio	descripcion	dia_detectada	hora	estado
1	41	Incendio	Edificio A	Rolando Velasco	Humo detectado cerca del la...	2026-09-09	10:30:00	ACTIVA
2	42	Medica	Edificio B	María Fernández	Estudiante herido en la canc...	2026-09-01	04:30:00	ACTIVA

Figura 7 Consulta SELECT utilizado WHERE, para la práctica de clase 1

5.3.4. Consulta 4: Utilizando UPDATE, modificar el estado las emergencias a “Atendida” donde el estado sea igual a “Activa”.

```

46
47 UPDATE public.alertas SET estado = 'ATENDIDA'
48 WHERE estado = 'ACTIVA';
49

```

UPDATE 2

Query returned successfully in 190 msec.

Figura 8 Consulta UPDATE, para modificar estados de emergencia.

Comprobación final de UPDATE: Finalmente se procedió a realizar un select para comprobar que los registros se hayan actualizado correctamente. La comprobación mostrando 0 registros con estado “Activa”.

```

49
50 SELECT * FROM public.alertas
51 WHERE estado = 'Activa';
52

```

	id_alerta [PK]	id_usuario	id_tipo	id_ubicacion	comando_recibido	descripcion	estado	hora_emerg	fecha
	integer	integer	integer	integer	character varying (50)	character varying (150)	character varying (50)	time without time zone	date

Figura 9 Consulta de verificación de datos actualizados de UPDATE.

5.3.5. Consulta 5: Utilizando UPDATE, cambiar el usuario de la emergencia, donde su estado sea igual a "EN_PROCESO".

```
54
55 UPDATE public.alertas SET id_usuario = 1
56 WHERE estado = 'EN_PROCESO';
57
```

Data Output Messages Notifications

UPDATE 1

Query returned successfully in 144 msec.

Figura 10 Consulta UPDATE para modificar el usuario de una alerta

Comprobación final de UPDATE: Finalmente se procedió a realizar un select, para cerciorarse de que el cambio de usuario haya sido exitoso. La prueba fue exitosa mostrando que la emergencia con estado “En_procceso” fue cambiado correctamente.

```
57
58 SELECT * FROM public.alertas
59 WHERE estado = 'EN_PROCESO';
60
```

Data Output Messages Notifications

Showing rows: 1 to 1 Page No: 1 of 1

	id_alerta [PK] integer	id_usuario integer	id_tipo integer	id_ubicacion integer	comando_recibido character varying (50)	descripcion character varying (150)	estado character varying (50)	hora_emerg time without time zone	fecha date
1	43	1	3	3	Seguridad,Edif_C,Activa	Persona no autorizada detec...	EN_PROCESO	11:45:00	2026-09-...

Figura 11 Consulta de verificación de datos actualizados

5.3.6. Consulta 6: Utilizando DELETE, eliminar las emergencias donde su estado sea igual a “Falsa”.

Para realizar esta sentencia, se hizo uso de DELETE, utilizando una condición mediante WHERE para eliminar solo los registros deseados, con esto se evita eliminar todos los registros de la tabla alertas.

```
63
64 DELETE FROM public.alertas
65 WHERE estado = 'FALSA';
66
```

Data Output Messages Notifications

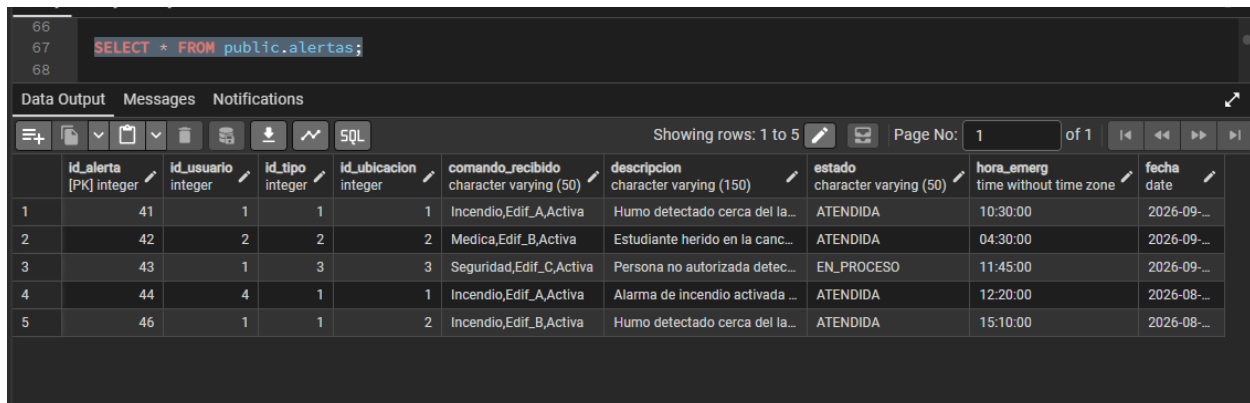
DELETE 2

Query returned successfully in 182 msec.

Figura 12 Consulta DELETE para la eliminación de alerta con estado Falsa

Comprobación final de DELETE: Una vez ejecutada la sentencia DELETE correctamente, se procedió a realizar una consulta de la tabla para determinar si la eliminación de las emergencias

fue correcta. La prueba resultó exitosa mostrando que se eliminó las emergencias con estado “Falsa”.



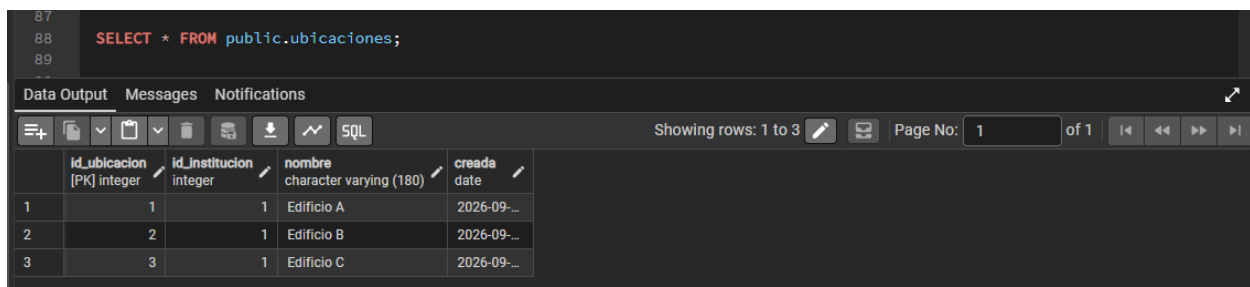
The screenshot shows a database query interface with a SQL editor at the top containing the query: `SELECT * FROM public.alertas;`. Below the editor, the 'Data Output' tab is active, displaying a table with 10 columns and 5 rows of data. The columns are: `id_alerta` (integer, PK), `id_usuario` (integer), `id_tipo` (integer), `id_ubicacion` (integer), `comando_recibido` (character varying (50)), `descripcion` (character varying (150)), `estado` (character varying (50)), `hora_emerg` (time without time zone), and `fecha` (date). The rows represent emergency records with details like location, command received, description, status, and time.

	<code>id_alerta</code> [PK] integer	<code>id_usuario</code> integer	<code>id_tipo</code> integer	<code>id_ubicacion</code> integer	<code>comando_recibido</code> character varying (50)	<code>descripcion</code> character varying (150)	<code>estado</code> character varying (50)	<code>hora_emerg</code> time without time zone	<code>fecha</code> date
1	41	1	1	1	Incendio,Edif_A,Activa	Humo detectado cerca del la...	ATENDIDA	10:30:00	2026-09-...
2	42	2	2	2	Medica,Edif_B,Activa	Estudiante herido en la canc...	ATENDIDA	04:30:00	2026-09-...
3	43	1	3	3	Seguridad,Edif_C,Activa	Persona no autorizada detec...	EN_PROCESO	11:45:00	2026-09-...
4	44	4	1	1	Incendio,Edif_A,Activa	Alarma de incendio activada ...	ATENDIDA	12:20:00	2026-08-...
5	46	1	1	2	Incendio,Edif_B,Activa	Humo detectado cerca del la...	ATENDIDA	15:10:00	2026-08-...

Figura 13 Consulta de verificación de DELETE.

5.4. CONSULTAS: CLASE 2

5.4.1. Consulta 1: Mostrar todos los registros de una tabla: Para realizar esta consulta se utilizó la tabla instituciones, la cual utilizando la sentencia SELECT fue posible obtener las ubicaciones registradas de la institución educativa.



The screenshot shows a database query interface with a SQL editor at the top containing the query: `SELECT * FROM public.ubicaciones;`. Below the editor, the 'Data Output' tab is active, displaying a table with 4 columns and 3 rows of data. The columns are: `id_ubicacion` (integer, PK), `id_institucion` (integer), `nombre` (character varying (180)), and `creada` (date). The rows represent building locations within an institution.

	<code>id_ubicacion</code> [PK] integer	<code>id_institucion</code> integer	<code>nombre</code> character varying (180)	<code>creada</code> date
1	1	1	Edificio A	2026-09-...
2	2	1	Edificio B	2026-09-...
3	3	1	Edificio C	2026-09-...

Figura 14 Consulta SELECT de la tabla Ubicaciones

5.4.2. Consulta 2: Filtrar información utilizando WHERE. En esta consulta se utilizó “where” con la condición de mostrar todos los registros de la tabla “alertas” donde su estado sea igual “En_proceso”.

```

94  SELECT
95      alertas.id_alerta,
96      tipos_emergencia.tipo AS emergencia,
97      ubicaciones.nombre AS ubicacion,
98      usuarios.nombre AS usuario_atendio,
99      alertas.descripcion AS descripcion,
100     alertas.fecha AS dia_detectada,
101     alertas.estado
102
103  FROM public.alertas
104  LEFT JOIN public.usuarios ON alertas.id_usuario = usuarios.id_usuario
105  INNER JOIN public.tipos_emergencia ON alertas.id_tipo = tipos_emergencia.id_tipo
106  INNER JOIN public.ubicaciones ON alertas.id_ubicacion = ubicaciones.id_ubicacion
107  WHERE alertas.estado = 'EN_PROCESO';

```

id_alerta	emergencia	ubicacion	usuario_atendio	descripcion	dia_detectada	estado
50	Seguridad	Edificio C	Juan Pablo Rivas	Persona no autorizada detecta...	2026-09-02	EN_PROCESO

Figura 15 Consulta de SELECT utilizando la condición WHERE

5.4.2. Consulta 3: Buscar información utilizando otro criterio. Para realizar una consulta utilizando otro criterio se utilizó WHERE IN (), para mostrar todas las emergencias, donde su estado sea igual a “Atendida o En_proceso”.

```

112  SELECT
113      alertas.id_alerta,
114      tipos_emergencia.tipo AS emergencia,
115      ubicaciones.nombre AS ubicacion,
116      usuarios.nombre AS usuario_atendio,
117      alertas.descripcion AS descripcion,
118      alertas.fecha AS dia_detectada,
119      alertas.estado
120
121  FROM public.alertas
122  LEFT JOIN public.usuarios ON alertas.id_usuario = usuarios.id_usuario
123  INNER JOIN public.tipos_emergencia ON alertas.id_tipo = tipos_emergencia.id_tipo
124  INNER JOIN public.ubicaciones ON alertas.id_ubicacion = ubicaciones.id_ubicacion
125  WHERE alertas.estado IN ('ATENDIDA', 'EN_PROCESO');
126

```

id_alerta	emergencia	ubicacion	usuario_atendio	descripcion	dia_detectada	estado
53	Incendio	Edificio B	Rolando Velasco	Humo detectado cerca del lab...	2026-08-28	ATENDIDA
51	Incendio	Edificio A	Ana Lucía Gómez	Alarma de incendio activada e...	2026-08-30	ATENDIDA
50	Seguridad	Edificio C	Rolando Velasco	Persona no autorizada detecta...	2026-09-02	EN_PROCESO

Figura 16 Consulta de SELECT utilizando WHERE IN().

5.4.2. Consulta 4: Ordenar información utilizando ORDER BY. En esta consulta se utilizó “Order by”, para ordenar las emergencias por su “fecha de creación” descendientemente, es decir de la más reciente a la más antigua.

129	SELECT
130	alertas.id_alerta,
131	tipos_emergencia.tipo AS emergencia,
132	ubicaciones.nombre AS ubicacion,
133	alertas.descripcion AS descripcion,
134	alertas.fecha AS dia_detectada,
135	alertas.hora_emerg AS hora,
136	alertas.estado
137	
138	FROM public.alertas
139	LEFT JOIN public.usuarios ON alertas.id_usuario = usuarios.id_usuario
140	INNER JOIN public.tipos_emergencia ON alertas.id_tipo = tipos_emergencia.id_tipo
141	INNER JOIN public.ubicaciones ON alertas.id_ubicacion = ubicaciones.id_ubicacion
142	ORDER BY alertas.fecha DESC;

Data Output								Messages	Notifications
								SQL	
Showing rows: 1 to 5								Page No: 1	of 1
	id_alerta	emergencia	ubicacion	descripcion	dia_detectada	hora	estado		
	integer	character varying (100)	character varying (180)	character varying (150)	date	time without time zone	character varying (50)		
1	48	Incendio	Edificio A	Humo detectado cerca del lab...	2026-09-09	10:30:00	ACTIVA		
2	50	Seguridad	Edificio C	Persona no autorizada detecta...	2026-09-02	11:45:00	EN_PROCESO		
3	49	Medica	Edificio B	Estudiante herido en la cancha...	2026-09-01	04:30:00	ACTIVA		
4	51	Incendio	Edificio A	Alarma de incendio activada e...	2026-08-30	12:20:00	ATENDIDA		
5	53	Incendio	Edificio B	Humo detectado cerca del lab...	2026-08-28	15:10:00	ATENDIDA		

Figura 17 Consulta SELECT utilizando ORDER BY para ordenar alertas

6. CONSULTAS SQL

A continuación, se presentan 5 consultas utilizando SELECT estructuras para obtener datos complejos y organizados de cada emergencia, que le dan respuesta a necesidades reales que el sistema solicita para su correcto funcionamiento.

6.1. Consulta 1 necesidad real del proyecto: ¿Cómo puedo buscar las emergencias por incendio mediante su descripción?

Para responder a esta necesidad, se utiliza la cláusula LIKE, la cual permite buscar un texto específico dentro del campo “descripción” de la tabla alertas. En este caso, la consulta busca las palabras clave “Humo” e “Incendio”, ya que pueden aparecer en la descripción de una emergencia relacionada con este tipo de evento.

147

148

149

150

151

152

153

154

155

156

157

158

159

160

161

```
SELECT
    alertas.id_alerta,
    tipos_emergencia.tipo AS emergencia,
    ubicaciones.nombre AS ubicacion,
    usuarios.nombre AS usuario_atendio,
    alertas.descripcion AS descripcion,
    alertas.fecha AS dia_detectada,
    alertas.estado

FROM public.alertas
LEFT JOIN public.usuarios ON alertas.id_usuario = usuarios.id_usuario
INNER JOIN public.tipos_emergencia ON alertas.id_tipo = tipos_emergencia.id_tipo
INNER JOIN public.ubicaciones ON alertas.id_ubicacion = ubicaciones.id_ubicacion
WHERE alertas.descripcion LIKE '%Humo%' OR alertas.descripcion LIKE '%Incendio%';
```

Data Output

Messages

Notifications

Showing rows: 1 to 2

Page No: 1 of 1

id_alerta	emergencia	ubicacion	usuario_atendio	descripcion	dia_detectada	estado
integer	character varying (100)	character varying (180)	character varying (100)	character varying (150)	date	character varying (50)
1	48	Incendio	Edificio A	Rolando Velasco	Humo detectado cerca del lab...	2026-09-09
2	53	Incendio	Edificio B	Rolando Velasco	Humo detectado cerca del lab...	2026-08-28

Figura 18 Consulta SELECT utilizando LIKE

6.2. Consulta 2 necesidad real del proyecto: ¿Cuáles son las emergencias registradas del edificio “A”?

Esta es una necesidad real dentro del sistema, ya que es necesario poder consultar las emergencias registradas en cada uno de los edificios de la institución. Para realizar esta consulta se utiliza la condición WHERE, con la finalidad de filtrar los registros de acuerdo con su ubicación.

En este caso, se establece como condición que la ubicación de la emergencia sea igual a “Edificio A”. De esta manera, la consulta devuelve únicamente los registros de la tabla alertas cuya ubicación coincida con ese edificio.

100

167

168

169

170

171

172

173

174

175

176

177

178

179

SELECT

alertas.id_alerta,

tipos_emergencia.tipo AS emergencia,

ubicaciones.nombre AS ubicacion,

alertas.descripcion AS descripcion,

alertas.estado

FROM public.alertas

LEFT JOIN public.usuarios ON alertas.id_usuario = usuarios.id_usuario

INNER JOIN public.tipos_emergencia ON alertas.id_tipo = tipos_emergencia.id_tipo

INNER JOIN public.ubicaciones ON alertas.id_ubicacion = ubicaciones.id_ubicacion

WHERE ubicaciones.nombre = 'Edificio A';

Data Output

Messages

Notifications

Showing rows: 1 to 2

Page No: 1 of 1

1

2

id_alerta

emergencia

ubicacion

descripcion

estado

integer

character varying (100)

character varying (180)

character varying (150)

character varying (50)

1

48

Incendio

Edificio A

Humo detectado cerca del lab...

ACTIVA

2

51

Incendio

Edificio A

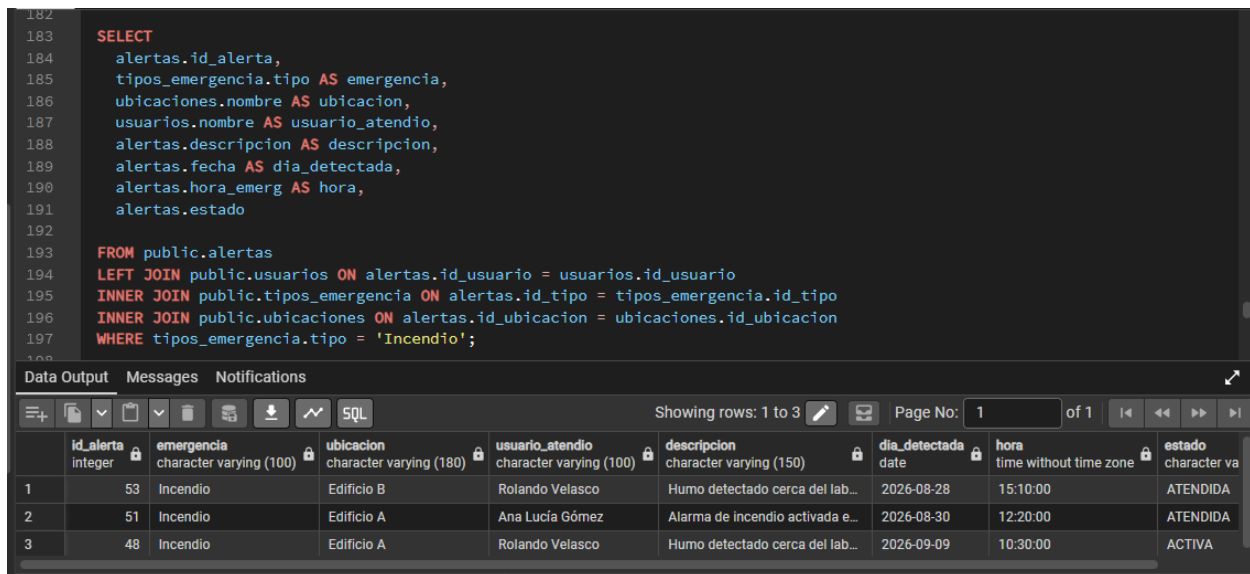
Alarma de incendio activada e...

ATENDIDA

Figura 19 Consulta para mostrar alertas por ubicación.

6.3. Consulta 3 necesidad real del proyecto: ¿Cuáles son las emergencias de tipo incendio registradas en sistema?

Para dar respuesta a esta pregunta, se realiza una consulta utilizando LEFT JOIN e INNER JOIN, para obtener los datos más completos de cada emergencia almacenados en diferentes tablas. Luego, se utiliza la condición WHERE para seleccionar todas las emergencias cuyo tipo sea igual a "Incendio".



```
182
183 SELECT
184     alertas.id_alerta,
185     tipos_emergencia.tipo AS emergencia,
186     ubicaciones.nombre AS ubicacion,
187     usuarios.nombre AS usuario_atendio,
188     alertas.descripcion AS descripcion,
189     alertas.fecha AS dia_detectada,
190     alertas.hora_emerg AS hora,
191     alertas.estado
192
193 FROM public.alertas
194 LEFT JOIN public.usuarios ON alertas.id_usuario = usuarios.id_usuario
195 INNER JOIN public.tipos_emergencia ON alertas.id_tipo = tipos_emergencia.id_tipo
196 INNER JOIN public.ubicaciones ON alertas.id_ubicacion = ubicaciones.id_ubicacion
197 WHERE tipos_emergencia.tipo = 'Incendio';
```

Data Output Messages Notifications

Showing rows: 1 to 3 of 1

	id_alerta integer	emergencia character varying (100)	ubicacion character varying (180)	usuario_atendio character varying (100)	descripcion character varying (150)	dia_detectada date	hora time without time zone	estado character va
1	53	Incendio	Edificio B	Rolando Velasco	Humo detectado cerca del lab...	2026-08-28	15:10:00	ATENDIDA
2	51	Incendio	Edificio A	Ana Lucía Gómez	Alarma de incendio activada e...	2026-08-30	12:20:00	ATENDIDA
3	48	Incendio	Edificio A	Rolando Velasco	Humo detectado cerca del lab...	2026-09-09	10:30:00	ACTIVA

Figura 20 Consulta para obtener registros de tipo "Incendio".

6.4. Consulta 4 necesidad real del proyecto: ¿Cuántas emergencias hay por edificio registradas?

Para dar respuesta a esta pregunta, se realiza una consulta utilizando GROUP BY. Gracias a esta sentencia SQL, es posible agrupar los registros que tienen la misma ubicación. De esta manera, se pueden contar las emergencias registradas en cada edificio y conocer cuántas corresponden a cada ubicación.

```

211
212 SELECT
213     tipos_emergencia.tipo AS emergencia,
214     ubicaciones.nombre AS ubicacion,
215     COUNT(tipos_emergencia.tipo) AS cantidad_emergencias
216
217 FROM public.alertas
218 LEFT JOIN public.tipos_emergencia ON alertas.id_tipo = tipos_emergencia.id_tipo
219 INNER JOIN public.ubicaciones ON alertas.id_ubicacion = ubicaciones.id_ubicacion
220 GROUP BY (ubicaciones.nombre, tipos_emergencia.tipo);
221

```

	emergencia character varying (100)	ubicacion character varying (180)	cantidad_emergencias bigint
1	Incendio	Edificio A	2
2	Incendio	Edificio B	1
3	Medica	Edificio B	1
4	Seguridad	Edificio C	1

Figura 21 Consulta SELECT utilizando GROUP BY

6.5. Consulta 5 necesidad real del proyecto: ¿Cuántas emergencias se han registrado de cada tipo?

Para dar respuesta a esta pregunta, se realiza una consulta utilizando GROUP BY y COUNT, agrupando las emergencias según su tipo. De esta manera, se puede conocer cuántas emergencias se han registrado de cada tipo, como incendio, médica y seguridad.

```

200
201 SELECT
202     tipos_emergencia.tipo AS emergencia,
203     COUNT(tipos_emergencia.tipo) AS cantidad_emergencias
204
205 FROM public.alertas
206 LEFT JOIN public.tipos_emergencia ON alertas.id_tipo = tipos_emergencia.id_tipo
207 INNER JOIN public.ubicaciones ON alertas.id_ubicacion = ubicaciones.id_ubicacion
208 GROUP BY (tipos_emergencia.tipo);
209

```

	emergencia character varying (100)	cantidad_emergencias bigint
1	Incendio	3
2	Medica	1
3	Seguridad	1

Figura 22 Consulta SELECT para mostrar cantidad de emergencias por tipo con GROUP BY

7. TRABAJO COLABORATIVO CON GITHUB

7.1 Repositorio

Para el desarrollo de las actividades de esta semana y entregas futuras, se creó el repositorio público titulado "SafeSchool-Alert-BD", el cual se estuvo trabajando en colaboración con el equipo de trabajo.

Este repositorio será el que se utiliza para organizar y gestionar de manera colaborativa los archivos de la base de datos relacional (desarrollada en PostgreSQL) del sistema SafeSchool Alert, para ello se estructuró el proyecto carpetas de documentación, modelo, evidencia, scripts del proyecto y finalmente un README, para mostrar información importante del repositorio y el proyecto, así como los métodos de instalación del repositorio etc.

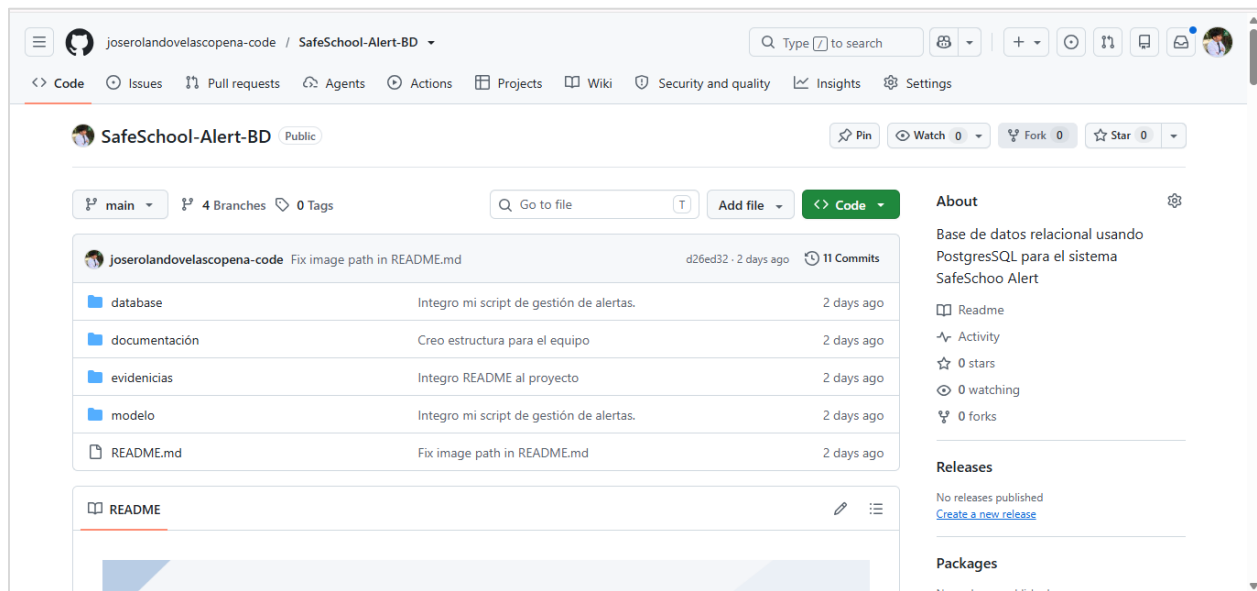


Figura 23 Captura del repositorio utilizado en GitHub

7.2 Rama individual

A continuación, se presenta la estructura del repositorio público titulado "SafeSchool-Alert-BD", el cual fue creado y trabajado en colaboración con el equipo del proyecto. La creación de esta rama me permitió organizar, trabajar de manera individual las responsabilidades que se me asignaron, la cual era trabajar y gestionar la tabla alertas, de este modo fue posible trabajar

con mis scripts, la documentación, los modelos y las evidencias de la base de datos relacional en PostgreSQL del sistema SafeSchool Alert, para luego ser integrado a la rama “main” el trabajo realizado en la rama, de acuerdo con las actividades de esta semana.

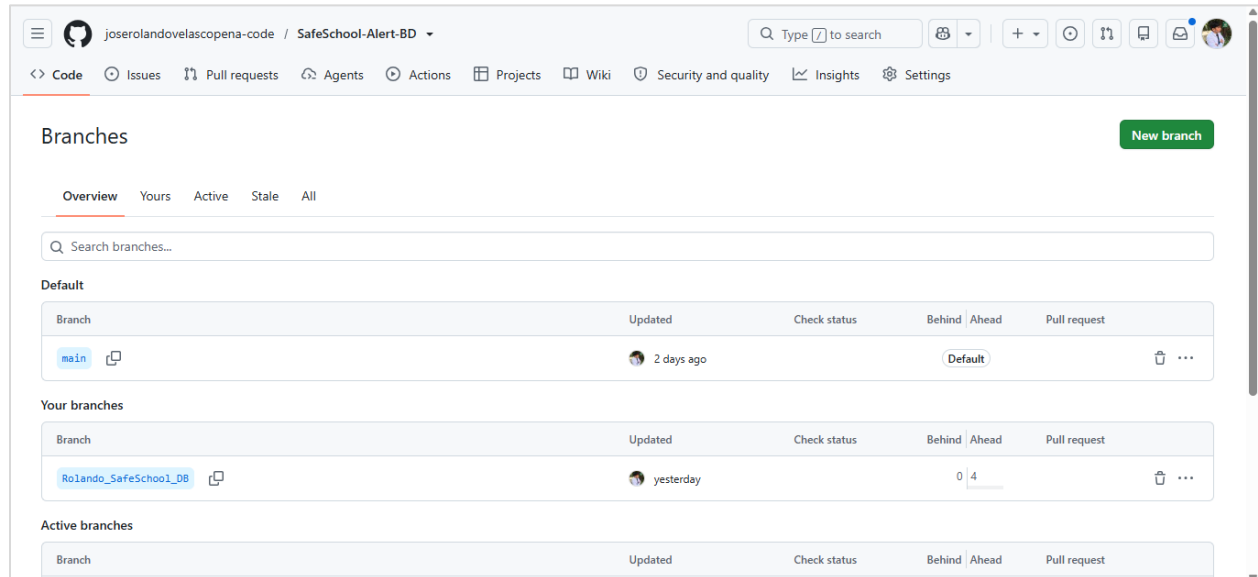


Figura 24 Captura de rama utilizada “Rolando_SafeSchool_DB”.

8. COMMITS

En esta captura se muestran los commits realizados en la rama main del repositorio de SafeSchool Alert, donde se evidencia mi participación individual en el desarrollo del proyecto. Entre los cambios realizados se encuentran la actualización de la estructura de la base de datos, la integración del script de gestión de alertas y la integración del README del proyecto del proyecto. También se muestra los pull requests utilizados para integrar los cambios realizados en mi rama de trabajo.

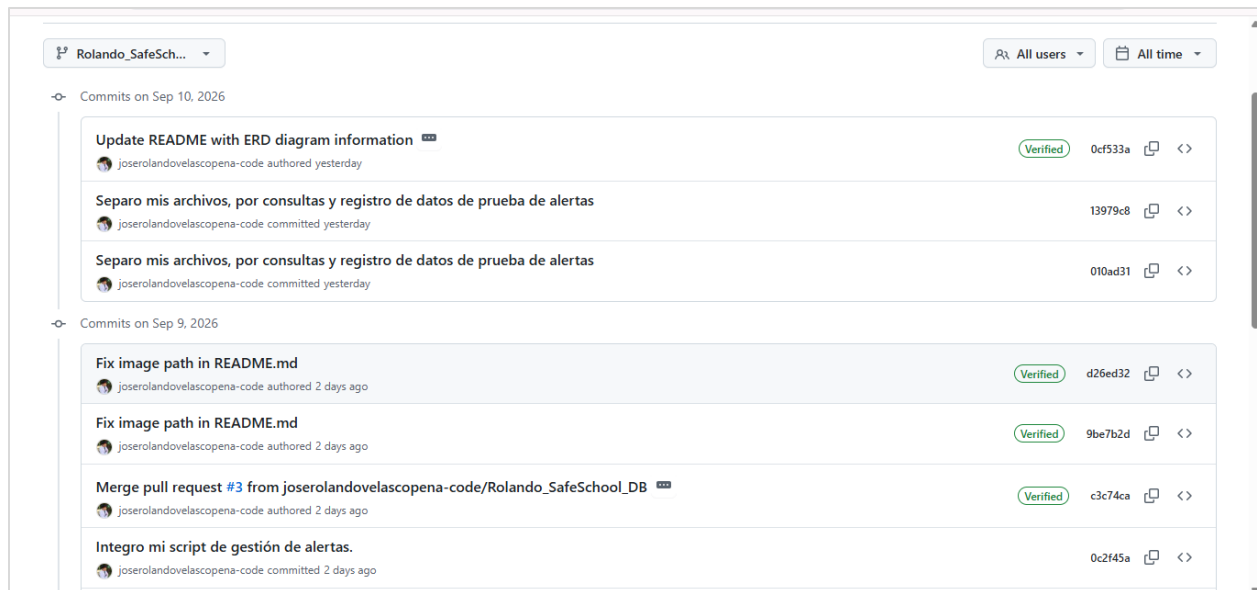


Figura 25 Captura de evidencia del Historial de COMMITS realizado

9. INTEGRACIÓN DEL APORTE

9.1. Historial de COMMITS

Finalmente, se puede observar el historial de commits realizados en la rama principal (main), donde se muestra la integración del trabajo realizado mediante un Pull Request. En esta actualización se agregaron los scripts 02_datos_prueba_alertas.sql y 03_consultas_alertas.sql, comprobando que los datos de prueba y las consultas realizadas fueron integrados correctamente en la base de datos de SafeSchool Alert. Con esto, se evidencia mi participación durante esta semana en el desarrollo del proyecto, trabajando de manera colaborativa y aportando al desarrollo de la base de datos.

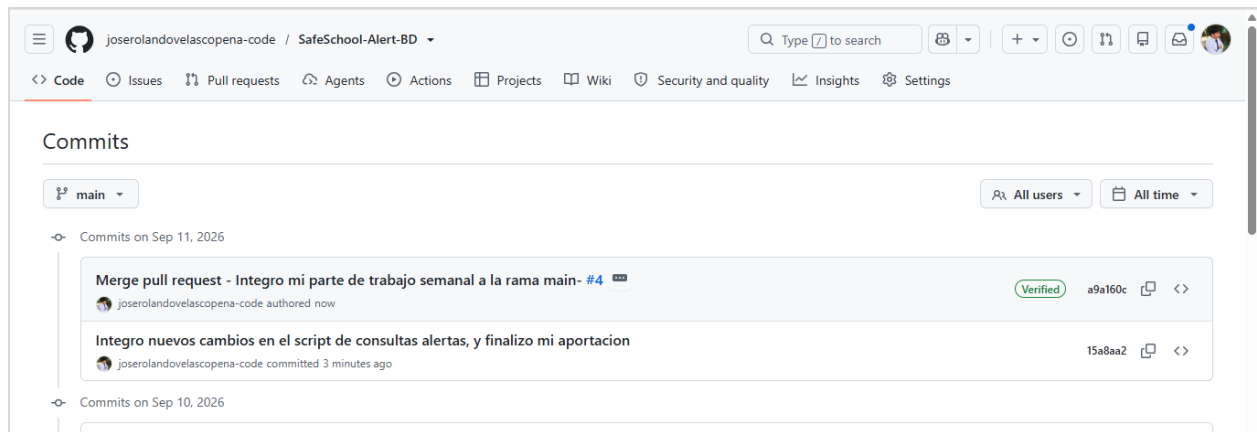


Figura 26 Capturad de evidencia de integración del mi trabajo a la rama main

9.2. Archivos modificados

A continuación, se presentan los cambios y archivos que estuvieron a mi cargo durante el desarrollo del proyecto. En la captura se puede observar los cambios realizados dentro de la carpeta database/, donde se crearon y editaron los scripts 02_datos_prueba_alertas.sql y 03_consultas_alertas.sql.

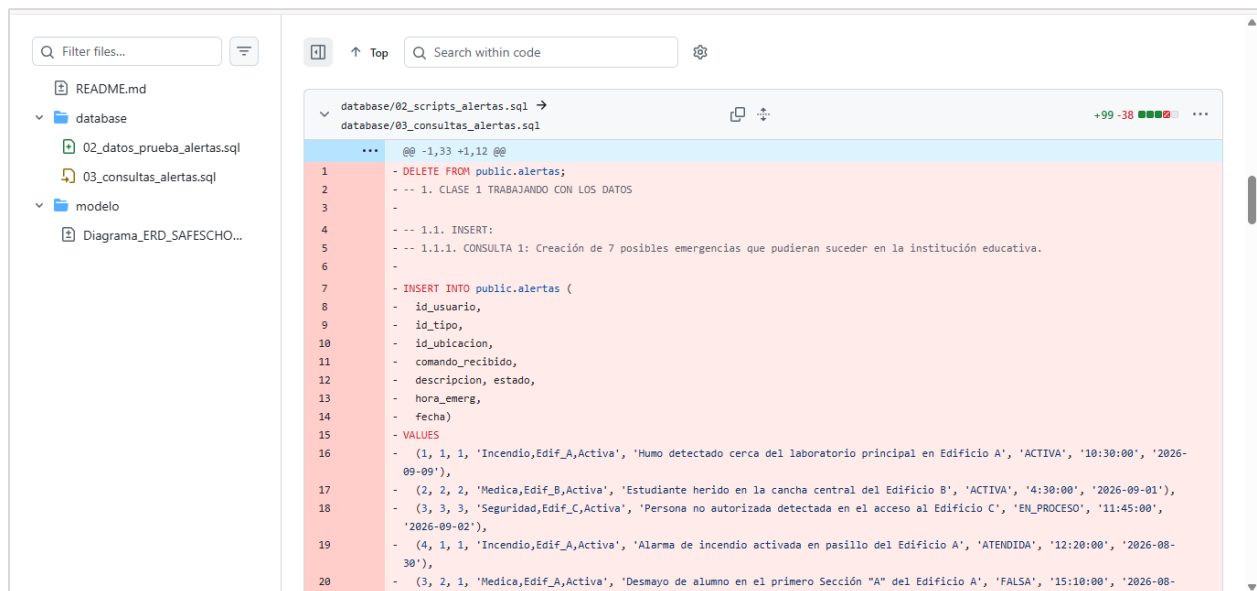


Figura 27 Captura de archivos modificados

10. REFLEXIÓN INDIVIDUAL

10.1 ¿Qué operación SQL aprendí a utilizar mejor?

Durante esta práctica aprendí a manejar y entender mejor las sentencias SELECT, integrándolas con LEFT JOIN e INNER JOIN. Mediante las consultas realizadas pude entender cómo obtener datos relacionados de diferentes tablas y mostrarlos en una sola consulta. Además, aprendí a utilizar de mejor manera GROUP BY para agrupar los datos y mostrar cantidades de emergencias por edificio y por tipo de emergencia.

10.2 ¿Qué consulta desarrollé para solucionar una necesidad de mi proyecto?

La consulta que desarrollé fue mediante SELECT, ya que me permite obtener información importante de las emergencias registradas en el sistema. Con estas consultas puedo conocer el estado, tipo y edificio donde ocurrió cada emergencia, facilitando la gestión y el seguimiento de las alertas detectadas por SafeSchool Alert.

10.3 ¿Qué aporte realicé al trabajo de mi equipo?

Aporté al proyecto y a mi equipo desarrollando consultas SQL para obtener información de las emergencias detectadas por el sistema. Mediante las consultas realizadas con SELECT, ayudé a obtener los diferentes estados de las emergencias y mostrar información más detallada de cada una, contribuyendo al funcionamiento y organización de la base de datos.

10.4 ¿Qué aprendí al utilizar ramas y commits en GitHub?

Durante esta práctica aprendí a utilizar de mejor manera las ramas y los commits en GitHub para trabajar de forma más ordenada con mi equipo. También comprendí que los commits permiten llevar un registro de los cambios realizados y que mediante las ramas y los Pull Request se puede integrar el trabajo de cada integrante sin afectar directamente el trabajo de los demás.

10.5 ¿Qué dificultad tuve y cómo la solucioné?

Una de las dificultades que tuve fue comprender cómo realizar correctamente las consultas utilizando JOIN y GROUP BY, ya que al principio no tenía claro cómo relacionar los datos de las diferentes tablas. Para solucionarlo, fui realizando las consultas paso a paso y

revisando los resultados mediante SELECT, hasta comprobar que los datos obtenidos fueran correctos. Esto me ayudó a comprender mejor cómo relacionar y organizar la información de la base de datos.

11. CONCLUSIÓN

En conclusión, esta práctica me permitió mejorar mis conocimientos sobre las consultas SQL. También aprendí a relacionar y organizar mejor los datos de las emergencias. Pude comprender de mejor manera el uso de SELECT, JOIN y GROUP BY. Además, aprendí a utilizar de mejor manera la herramienta de almacenamiento de código GitHub mediante ramas, commits y Pull Requests utilizando Git para llevar un control de las versiones de los cambios y modificaciones de los scripts utilizados para cumplir con las actividades de esta semana y Pull Request. Todo esto me permitió aportar al proyecto y trabajar de manera colaborativa con mi equipo.